REPORT · 2026年10月

交付评估、GUI 诊断与资源清理

中小型文物三维建模自动化系统 v1.1.6 交付审计

文保数字化研发组
2026年10月

目 录

一、结论摘要 3

二、本次建模任务评估 4

2.1 交付物核对 4

2.2 耗时结构 4

2.3 几何质量 4

2.4 纹理质量 5

2.5 一个必须修正的口径：中误差 41.16 不是毫米 6

2.6 达标判定 6

2.7 评估结论 6

三、GUI 问题分析与修复 8

3.1 事故复现：一次正确的重跑被报成失败 8

3.2 问题清单 8

3.3 三条根因 9

3.4 已实施的修复 9

3.5 验证 10

3.6 仍未处理与建议 10

四、可清理资源清单 11

4.1 总量 11

4.2 E:\h3d_lablab 可删清单 11

4.3 E:\兵马俑照片 可删清单 11

4.4 建议执行顺序 12

五、后续建议 13

一、结论摘要

本报告对三个问题给出结论：**（1）** 本次在 `E:\兵马俑照片\最终测试` 上完成的建模任务究竟做到了什么水平；**（2）** 为什么「管线跑完之后再点一次『一键生成交付物』会报错」——这不是偶发故障，而是三条独立的设计缺陷叠加；**（3）** 两个工作目录里哪些东西可以安全删除，删了能回收多少空间。

> **一句话结论**：本次建模**技术上成立、可作为论文实证案例**（3 种格式全部产出，贴图岛内填充率 94.6%，30 台相机全部参与投影）；但报表里的**中误差 41.16 不能当毫米读**，它是 0.041 模型单位（对角线 0.68%）。GUI 的报错源于「跳过被当成失败 + 无重入闸门 + 打开项目丢配置」，三者均已修复并通过 717 项测试。两个目录合计可安全回收约 **21 GB**。

本次交付的完整链路耗时 1866.7 s（31 min 07 s），其中 S1 独占 1784.0 s（95.6%），而 S1 内部稠密重建又占 77%。扣除 S1，全流程只剩 82.7 s，其中 M4 贴图 73.5 s（88.9%）。这个结构与 v1.0 系列相比已经收敛——没有哪一处再出现「几十倍于设计值」的异常。

本次是**单件首次解算**，因此 S1 照片指纹复用与 M4 图集复用都没有命中（`atlas_reuse.state=miss`）。若同一批照片再跑第二件，S1 降至 0.2~0.3 s、M4 降至 0.5~0.6 s，整批时间由约 31 min 压到约 2 min。换句话说：**本次的 31 分钟是一次性的必要成本，不是管线的稳态成本。**

二、本次建模任务评估

2.1 交付物核对

M6 发布了 3 种格式，全部落在 80_export/relic_BMY/，并附带贴图、精度表与论文表格。三种格式的几何与纹理同源（同一 M3 网格 + 同一 2048 图集），差别只在容器与压缩方式。

产物	大小	说明
relic_BMY.glb	7.83 MB	单文件自包含，二进制膨胀最大（顶点色 + 纹理 + 索引）
relic_BMY.obj + .mtl	13.86 MB	文本格式，map_Kd 指向同目录贴图
relic_BMY.ply	5.92 MB	顶点色回退（PLY 不支持纹理）
relic_BMY_diffuse.png	4.70 MB	2048 图集，与 50_tex 同名贴图一致
accuracy.csv / thesis_table.csv	1 KB 级	精度指标与论文表格
report/	3 文件	CSV + 耗时图 + HTML 报表

合计 **31 MB / 件。**这个体积对论文附件与交付光盘都合适；真正占空间的是中间产物（本次 99_runs/sfm_tmp 就有 1.7 GB），这也是第四章清理清单的重点。

2.2 耗时结构

阶段	耗时 (s)	占总时长	相对上一档	评价
S1 稀疏（特征/匹配/建图）	411.2	22.0%	—	30 张 4496×3000，exhaustive 匹配，正常
S1 稠密（去畸变/深度图/融合）	1372.6	73.5%	3.3 倍	30 视图两遍 patch_match，单视图约 23 s
M1 背景剔除	5.832	0.31%	—	677k 点聚类 4.42 s（1.1.3 时曾 284 s）
M2 配准	0.032	0.002%	—	单站无配准（registered=否），空跑
M3 泊松网格	2.467	0.13%	—	41.5k 点建 117k 面，单线程，可复现
M4 照片贴图	73.514	3.94%	29.8 倍	UV 23.75 + 采样 48.34 + 导出 1.38
M5 质检	0.634	0.034%	—	4000 采样点
M6 发布	0.239	0.013%	—	3 格式落盘

两点值得写进论文：**第一**，S1 占了 95.6%，说明纯照片场景的成本几乎全在 SfM；M1~M6 六个阶段加起来只有 82.7 s，这正是「自动化管线」的价值所在——人工做这六步至少要半天。**第二**，用户提供的耗时图用的是对数坐标，红色柱标注「有失败」，但本次 **failed=0**，图上并没有红柱，说明这批图的标题还停留在旧版本的模板上，正式引用前应改成「本次无失败阶段」。

2.3 几何质量

指标	数值	读法
SfM 稠密点数（gate 前）	708,473	COLMAP fused.ply 原始点

指标	数值	读法
稠密质量门输出	677,263	剔除 4.41%（统计离群 11,430 + 密度 19,780）
M1 背景剔除后	622,192（91.87%）	78 个簇里只保留 1 个主簇，无平面被误删
降采样后送 M3	41,502	体素 0.02（模型单位），压缩到 6.67%
M3 顶点 / 三角面	58,990 / 117,449	面数约为点数 2.8 倍，属泊松的正常形态
连通块数	11	最大块占 **99.73%**，其余 10 块合计 0.27%
水密性	否	存在边界边，符合「单视角拍摄 + 泊松裁剪」的预期

几何的**主要风险不是碎片**（v1.1.3 之前最大块曾只有 27.8%，本次已到 99.73%），而是**绝对尺度不可知**：纯照片场景没有激光锚点，包围盒对角线 6.062 是任意单位。这一点 M1 与报表都已正确标注（`metric_units=否`），但下游指标的呈现仍需修正，见 2.5。

另一个数字值得注意：M1 保留了 91.87% 的点，但 M3 只用了 41,502 点。体素 0.02 相对于该模型（对角线 6.06 单位）约等于「每 300 分之一」的采样，对 58,990 顶点来说偏稀。若希望细节更足，可把体素降到 0.01 再跑一次 M3+M4，代价是面数翻约 3 倍（约 35 万面），M4 采样时间也会同比上升。

2.4 纹理质量

M4 是本次唯一「按设计耗时」的大阶段（73.5 s），结果可以逐项核对。三个比例指标含义不同，必须分开读：

指标	数值	判定依据
图集像素利用率 coverage	59.57%	有颜色的纹素 / 图集总面积（264 万 / 419 万）
UV 岛占比 atlas_utilization	62.99%	UV 岛面积 / 图集面积，由网格面积与展开决定
岛内填充率 island_fill	**94.57%**	coverage / atlas_utilization，**这才是「贴没贴上」的真口径**
参与相机	30 / 30	全部注册影像都参与投影，无被排除的相机
遮挡拒绝次数	7,059,987	逐纹素 Z-buffer 判定，量级与 30 视图 × 264 万纹素一致
背面填充纹素	612,816	无照片覆盖处用邻域扩散补齐，占已着色纹素的 23%
多视角色差 std	14.36	同一纹素跨 5 个视角的亮度标准差，中等偏低（说明曝光一致）
图集平均亮度	76.54	偏暗，符合陶俑本色，不是采样失败
像素换算 pixel_scale	1.8741	内参在 2400 长边、照片 4496 长边，比值 1.8733，一致

岛内填充率 94.57% 是本批最硬的指标：它说明 UV 岛内**只有 5.4% 的纹素没采到颜色**。v1.0 时代这个数字一度是 44.9%，根因是「每个面只溅射 4 个点」，现在逐纹素光栅化后已经收敛。covereage 59.57% 看着低，是因为图集本身留了空白边（UV 岛只占 63%），不是贴图质量问题——这一点报表第三节已经

写清楚了。

2.5 一个必须修正的口径：中误差 41.16 不是毫米

报表第四节写「中误差 41.16（任意单位）」，同时给出包围盒对角线 6.062。两个数放在一起是**自相矛盾**的：中误差不可能比整个模型的尺寸还大 6.8 倍。实际原因是 qc_rmse() 在返回前统一乘了 1000（源码注释是「米转毫米」）：

```
# src/h3d/quality/metrics.py
dists = np.asarray(pts.compute_point_cloud_distance(ref_pcd))
return float(np.sqrt(np.mean(np.square(dists))) * 1000.0)  # 米 -> 毫米
```

在激光场景下这个换算是正确的（输入单位就是米）。但纯照片场景的模型单位是任意单位，**乘 1000 等于把「1 个模型单位」当成「1 米」**。所以 41.16 的真实含义是 **0.04116 模型单位**，相对包围盒对角线只有 **0.68%**——这是一个相当好的结果，却被写成了「41 毫米」，足以让评审直接质疑。

读法	数值	是否成立
按报表字面读	41.16 mm	不成立（模型对角线只有 6.06「单位」）
按模型单位读	0.04116 单位	成立，相对对角线 0.68%
与 MPS 比较	0.04116 / 0.01705 = 2.41 倍点间距	成立，说明网格贴合参考点云的量级合理
rmse_reliable	False	M5 已正确识别「参考过疏，中误差不参与判门」

> **建议改法（未实施）**：metric_units=False 时，M5 应以 rmse_rel = rmse / bbox_diag 作为主指标（无量纲），报表中「中误差」一列显示 0.68%（0.041 单位） 而不是 41.16；只有 metric_units=True（有激光锚点/尺度控制点）时才显示 mm。这是论文可信度问题，不是显示问题。

2.6 达标判定

检查项	阈值	实测	结论
几何覆盖率	≥ 30%（本次配置）	83.63%	达标
边界边比例	越低越好	0.31%	达标（开放面少）
纹理岛内填充率	≥ 85%（内部口径）	94.57%	达标
纹理清晰度（拉普拉斯方差）	≥ 50	74.38	达标
纹理接缝（相邻岛梯度差）	≤ 5	2.33	达标
中误差	≤ 5 mm（仅米制场景）	见 2.5	本批不适用
P1 照片纹理独立阶段	—	未执行	本次走 M4 内置贴图，P1 未启用

整体判定 **pass**。需要说明的是「P1 未执行」不是缺陷：本项目的 photo_texture.enabled=false，照片贴图由 M4 内部的 paint_mode=photo 完成，两者产物等价，后者少一次 IO。报表把未执行阶段与失败阶段分开显示（「未执行」而非「0 个产物」），这个设计是对的。

2.7 评估结论

可以用于论文的三条实证：① 30 张 4496×3000 手持照片，全程零人工干预产出 PLY/OBJ/GLB 三种格式与完整质检报表；② 贴图岛内填充率 94.57%，说明自研 UV + Z-buffer + 多视图加权采样管线在真实数

据上成立；③ 除 S1 外六阶段合计 82.7 s，量化了「自动化」相对人工建模的时间收益。

必须补的三件事：① 中误差口径（2.5 节），否则数字站不住；② 若论文需要毫米级精度，必须补尺度控制点（X2）或激光扫描件，否则任何 mm 数值都只是内部自洽性；③ 镂空与手臂内侧缺照片覆盖，泊松会在这些位置留空洞，需补拍或接受并说明。

三、GUI 问题分析与修复

3.1 事故复现：一次正确的重跑被报成失败

用户在管线完成后再点一次「一键生成交付物」，界面弹出「运行未完成」。但实际日志（batch_gui.log）显示的是完全成功的续跑：七个阶段全部命中已有产物，**failed=0**，进程退出码 0。

```
[INFO] [batch_processor] 断点续跑：加载既有 manifest .../output/run_manifest.json
[INFO] [batch_processor] S1 跳过：relic_BMY（断点续跑，复用 2 个产物）
[INFO] [batch_processor] M1 跳过：relic_BMY（断点续跑，复用 1 个产物）
   ...（M2~M6 同样跳过）...
[INFO] [batch_processor] 批量处理完成：total=1 done=0 failed=0 skipped=1
[INFO] [report] 报表生成完成：3 个文件 -> .../report
```

GUI 的判定代码是 `ok = failed == 0 and done > 0`。「全部跳过」时 done=0，于是被判成失败。用本次的真实日志行做对照实验：

判据	对同一行日志的结果	界面表现
旧：`failed==0 and done>0`	**False**	弹「运行未完成」、进度条 100% 归零、状态变红
新：`failed==0 and done+skipped>0`	**True**	提示「无需重跑：全部 1 件已是最新」

这不是显示问题。**「跳过」与「失败」是两种终态**：跳过说明产物齐备、重算没有意义；失败说明产物缺失或损坏。把前者判成后者，用户会去删产物重跑，白白再花 30 分钟。

3.2 问题清单

顺着这条线索把 GUI 全量审了一遍，共 15 项，按后果分级。「阻断」= 会让用户做出错误决策或直接崩溃；「高」= 功能失效但可通过日志绕过；「中/低」= 体验与一致性问题。

问题	后果	级别	状态
1. 「全部跳过」被判成失败（`done>0` 判据）	幂等重跑弹失败框、进度归零	阻断	已修
2. 完成行之后不读剩余输出，也不等进程退出	报表还在写就说完成，此时再点可能撞数据根锁	高	已修
3. 无重入守卫，直接覆盖 `self.runner`	旧 QThread 被 GC → Qt 直接 abort	阻断	已修
4. 按钮禁用发生在开跑前自检之后	自检弹窗期间可重复点击	中	已修
5. 失败时进度条 `setValue(0)`	跑到 100% 却显示 0%，比不动更困惑	中	已修
6. 打开历史项目不读 `config.yaml`	只传 root/name/scene，配置对象是全新默认值	阻断	已修
7. `to_yaml` 无条件用模板重写整份配置	dense 由 true 变 false（直接跳过稠密），max_features 等回默认	阻断	已修
8. `quality.coverage_threshold` 硬编码写 0.30	用户调过的 0.80 被改回宽松档	中	已修
9. 接缝平滑 / 几何质检 / top-k / 每面采样 / 密度过滤是假控件	改了不生效，输出一模一样	高	已修

问题	后果	级别	状态
10. 「CUDA 加速」下拉框无后端开关	选「强制 CPU」不生效	低	改为禁用并说明
11. 取消按钮可重复点击	反复 taskkill 打空 PID，像卡住	低	已修
12. 每次点击清空日志	失败后回看上一轮日志已没了	低	已修
13. 完成后仍可点「一键生成」，无二次确认	误触即重跑（本次事故的触发条件）	高	改为幂等 + 明确提示
14. 结果用模态弹窗汇报	必须点掉才能继续，且无法回看	低	未修
15. 预览区有两个同名「3D 模型预览」标签页	第一个永远是空白死页	低	已修

3.3 三条根因

根因一：终态语义不完整。批处理有 4 种终态（完成 / 跳过 / 失败 / 未执行），而 GUI 只认 3 种，把「跳过」并进了「失败」。这个错误在日志解析层就有——`done_summary()` 只取 `total/done/failed` 三个字段，`skipped` 根本没被读进来。

根因二：配置只有「写出」没有「读回」。界面的配置流是单向的：控件 → `ProjectConfig` → 模板 → `config.yaml`。「打开历史项目」只把 root/name/scene 传进主窗口，配置对象是默认值，于是第一次点按钮就把整份 yaml 覆盖成模板。**用户调了三小时的参数，在点下按钮的那一秒丢失**，而界面上没有任何提示。

根因三：控件与配置没有契约。10 个控件里只有 5 个有回写通道。「接缝平滑」「几何质检」「视角加权 top-k」「每面采样点」「密度过滤」五项在界面上看起来完全正常，改了也「保存成功」，但它们从未进入 `config.yaml`。这类问题最隐蔽——不报错、不告警、结果不变，用户只会怀疑「算法不稳定」。

3.4 已实施的修复

文件	改动	锁定行为的测试
gui/logparse.py	新增 `batch_summary()` 返回含 skipped 的四元组；`done_summary()` 保留三元组以兼容旧调用；旧日志缺 skipped 时按 `total-done-failed` 兜底	6 项（含旧日志兜底、非完成行返回 None）
gui/runner.py	见到完成行只 `break` 不 `return`：读完余下 stdout 并 `wait()` 等进程退出；`ok` 改为 `failed==0 and rc==0 and done+skipped>0`；文案区分「无需重跑」	4 项（全跳过 / 部分跳过 / total=0 / 读完尾行）
gui/app.py	新增 `_busy()` 与 `_starting` 双重闸门；旧 runner 用 `deleteLater()` 释放；进度条不再归零；日志改为分隔线不清空；取消按钮点击即禁用；预览区删掉重复 tab	既有 96 项 GUI 测试全绿
gui/config_edit.py	新增 `ProjectConfig.from_root()` 读回 yaml；`to_yaml()` 以原文件为底稿只覆盖控件字段；`coverage_threshold` 改 `setdefault`；补 5 个此前只存在于界面的字段	5 项（字段回读 / 非控件字段保留 / 控件字段落盘 / 坏 yaml 容错 / 无 yaml）

文件	改动	锁定行为的测试
gui/advanced.py	apply() 由 5 个字段扩到 10 个，全部接通 to_yaml	由上面 5 项覆盖
gui/welcome.py	「打开历史项目」改用 from_root() 构造配置	同上

其中 from_root() 的改动最关键：它把配置流从「单向写出」改成「读改写」。用本次真实项目验证——读入 最终测试/config.yaml再写回，44 个字段里 **43 个逐值一致**，唯一差异是 quality.enabled 由「缺键」变成显式 true（等价默认值）。

3.5 验证

验证项	方法	结果
单元测试全量	pytest -q	**717 passed**（修复前 705，本批 +12）
真实日志复现	把 21:04 那份 batch_gui.log 的完成行喂给两个判据	旧 False / 新 True
配置保真	from_root → to_yaml 后与原文逐字段比对	44 字段，43 一致，1 处等价差异
配置未被 CLI 改写	源码跑一次 batch 后 diff config.yaml	完全一致
端到端跳过	源码 batch 跑本次项目	全 7 阶段跳过，末行 done=0 skipped=1，rc=0

需要明确一点：**以上修复都在源码里，打包版仍是 1.1.6 的行为。**要拿到可用的 exe，需要重新执行 make_version_info → PyInstaller → 建发行目录 → build_installer → --fill → sync_source_snapshot → make_release_zip（顺序不能变，快照必须最后同步）。

3.6 仍未处理与建议

项	建议	优先级
结果用模态弹窗汇报（#14）	改为界面内常驻「本次结果条」：阶段数、耗时、产物路径、报表按钮；弹窗只用于真正需要用户决策的场合	中
重复点击无二次确认（#13）	当前靠「幂等 + 提示无需重跑」兜底。更彻底的做法是给按钮加状态机：空闲 → 运行中（禁用）→ 已完成（改文案为「重新生成」并二次确认）	中
没有「强制重算」入口	界面应提供「清空本阶段产物后重跑」，而不是让用户自己去删目录（删错就废掉几十分钟的成果）	中
s1_reuse_index.json 只在同根生效	本次换了项目根（最终测试）就重新解了 30 分钟。建议索引支持跨根（按照片内容哈希全局查找）	低
M5 中误差口径（见 2.5）	纯照片场景改报无量纲相对值	高（论文相关）

给出一段自检口诀，供后续任何界面改动对照：**「每个控件的值有没有进配置文件？每个终态有没有被界面认出来？每个按钮连点两次会怎样？」**本次 15 个问题里有 11 个都能被这三问提前拦下。

四、可清理资源清单

4.1 总量

两个目录合计 27.6 GB。其中**真正的工程资产**（源码、测试、当前发行版、本次交付物）不到 2 GB，其余是构建中间目录、历史打包产物、COLMAP 稠密中间件与重复的照片副本。

目录	当前占用	可安全回收	回收后
E:\h3d_lablab	18.0 GB	约 13.6 GB	约 4.4 GB
E:\兵马俑照片	9.6 GB	约 7.7 GB	约 1.9 GB
合计	27.6 GB	**约 21.3 GB**	约 6.3 GB

4.2 E:\h3d_lablab 可删清单

按「删了会不会影响开发/能否重建」排序。前四项是**纯构建中间产物**，删掉后下次打包会重新生成，不影响任何代码与功能。

目录 / 文件	大小	是什么	为什么可删
build_work*（16 个）	4.29 GB	PyInstaller 工作目录（.toc/.pyz/缓存）	每次打包自动重建；不影响 exe
build_out*（12 个）	2.66 GB	历次打包产出的 H3D.exe（v1~v12）	当前版本已在 release\H3D_v1.1.6\
release/ 旧版本目录（1.1.0 / 1.1.4 / 1.1.5）	1.40 GB	历史发行目录	保留 H3D_v1.1.6 即可；如需回退点可留 1.1.5
release/ 旧 zip（1.1.0 ×2 / 1.1.4 / 1.1.5）	1.85 GB	历史发行压缩包	内容与对应目录重复
packaging/Output/	1.20 GB	Inno Setup 编译输出	Setup 已复制到 release
output/_build/dist_v6..v10	1.11 GB	早期打包试验产物	同上，已被 build_out 取代
output/ 下 19 个 _* 测试项目	1.75 GB	开发期端到端项目（_colmap_demo 732 MB 等）	测试用 tmp_path，无硬依赖
output/_old_H3D_r5.exe	227 MB	旧 exe 副本	已无用
output/_new_tex.npy 等零散大文件	15 MB	调试中间数组	可重算

> **必须保留**：src/、tests/、scripts/、configs/、docs/、assets/、packaging/（除 Output）、h3d_gui.spec、output/_build/ 里的 gen_*.py 与 render_pdf.py（PDF 生成链）、output/*.pdf（历史交付文档）、release/H3D_v1.1.6/ 与它的 zip、.git、.workbuddy/memory/。

4.3 E:\兵马俑照片 可删清单

这个目录有个结构问题值得先说：**它本身是一个项目根，里面又套了两个项目根**（最终测试、测试测试111），三层用的是同一批 30 张照片。结果是同一份照片被解了三次 SfM、存了 7 份副本。

目录 / 文件	大小	是什么	为什么可删
99_runs/sfm_tmp/	3.81 GB	5 件的 COLMAP 工作区（含两遍深度图）	交付物已在 10_dense/，中间件可重建

目录 / 文件	大小	是什么	为什么可删
最终测试/99_runs/sfm_tmp/	1.70 GB	本次项目的稠密中间件	同上；删除还能消除「位姿变了但深度图残留」的隐患
测试测试111/99_runs/sfm_tmp/	1.70 GB	早期测试项目中间件	同上
测试测试111/ 其余部分	0.20 GB	一次已结束的测试（relic_001）	与 最终测试 同源，结论已被后者取代
重复照片副本（5 件 + 2 单件，各 123 MB）	约 0.49 GB 冗余	同一批 DSC_0008~DSC_00xx 被复制 7 份	留 1 份即可
99_runs/repro_clean / repro_stale / verify_reset	127 MB	可复现性验证的对照实验目录	结论已进报告

根目录那 5 件（relic_001BMYyyyy 等）虽然几何质量一般，但它们是「多件批处理 + S1 复用」的**实测证据**，建议保留 80_export/ 与 report/，删除其余中间目录即可（约 0.9 GB）。

4.4 建议执行顺序

按「先删最确定、再删需要掂量的」分三批。每批删完立刻验证，不要一次性全删——一旦发现要回滚，分批还能保住大部分。**以下命令仅供你确认后自行执行，我没有删除任何文件。**

```
REM ===== 第 1 批：纯构建中间产物（约 8.1 GB，零风险）=====
cd /d E:/h3d_lablab
for /d %d in (build_work* build_out*) do rmdir /S /Q "%d"
rmdir /S /Q packaging\Output
for /d %d in (output\_build\dist_v*) do rmdir /S /Q "%d"
del /Q output\_old_H3D_r5.exe

REM ===== 第 2 批：历史发行版与测试项目（约 5.4 GB）=====
cd /d E:/h3d_lablab
rmdir /S /Q release\H3D_v1.1.0
rmdir /S /Q release\H3D_v1.1.4
del /Q release\H3D_v1.1.0_*.zip
del /Q release\H3D_v1.1.4_*.zip
del /Q release\H3D_v1.1.5_*.zip
for /d %d in (output\_*) do if /I not "%d"=="output\_build" rmdir /S /Q "%d"

REM ===== 第 3 批：COLMAP 中间件（约 7.2 GB，删后下次 S1 需重算）=====
rmdir /S /Q "E:/兵马俑照片/99_runs/sfm_tmp"
rmdir /S /Q "E:/兵马俑照片/最终测试/99_runs/sfm_tmp"
rmdir /S /Q "E:/兵马俑照片/测试测试111/99_runs/sfm_tmp"
```

> **删 sfm_tmp 前确认一件事**：10_dense/<item>/fused_dense.ply 与 90_meta/<item>_capture_meta.json 都还在。这两个是 S1 复用的**唯一**输入，缺一个就要重跑 30 分钟。删 sfm_tmp 反而消除了「位姿变了、深度图残留」导致点云错乱的隐患（自检日志里那条「1 件存在上次的深度图」提示就是它）。

另外提醒：**清理前后各跑一次 H3D.exe doctor --json**，确认 project 作用域没有新增阻断项。如果删 release 旧版本时误删了 H3D_v1.1.6，从 zip 解压回来即可（zip 里有完整目录结构）。

五、后续建议

立刻做：① 重新打包 v1.1.7 让 GUI 修复生效（本次 15 项里已改 12 项）；② 改 M5 中误差口径（2.5 节），这是论文里唯一一处「数字会被人抓住」的地方；③ 清理第 1 批（纯构建产物，8.1 GB，零风险）。

本周做：④ 用 v1.1.7 在同一批照片上跑第 2 件，验证 S1 复用与 M4 图集复用在真实多件项目里命中（预期整批 31 min 降至约 2 min）；⑤ 补拍镂空与手臂内侧，或明确在论文里说明「这些位置由泊松闭合，无真实纹理」。

论文层面：⑥ 把「跳过 / 完成 / 失败」三态语义与「幂等重跑」写进系统设计章节——这是可复现性的直接证据，也是本次事故反过来提供的素材；⑦ 若要做毫米级精度声明，必须补尺度控制点（X2）或激光扫描件，否则所有 mm 数值都只能作为内部自洽性指标。

最后一句实话：本次管线本身的表现是**合格**的，出问题的全是「管线之外的壳」——界面把正确的结果报成了错误。这类缺陷不会在日志里留下异常，只能靠「按钮连点两次会怎样」这种最朴素的测试去暴露。715 项单元测试没有一条覆盖它，这也是本批新增 12 项测试的由来。